

XRComm FCS 8T8R Platform

Application Development Guide

XRComm Inc.

June 2026

XRComm FCS 8T8R Platform

Application Development Guide

Platform	XRComm FCS 8T8R Platform
Driver Version	v1.4.3
Variant	8T8R
Audience	Software Developers
Classification	Proprietary

© 2026 XRComm Inc. – All Rights Reserved.

Contents

Introduction	3
Compiling and Running the Hello World Example	3
Compile the Example	3
Run the IQ Example	3
Verify the Application Is Connected	3
Application API Reference	4
Lifecycle and Connection Functions	4
xrcomm_ip_client_init	4
xrcomm_ip_client_init_port	4
xrcomm_ip_client_wait_ready	4
xrcomm_ip_client_running	4
xrcomm_ip_client_shutdown	4
Data Consumer Functions	4
xrcomm_ip_client_poll	4
xrcomm_ip_client_consume	4
Diagnostics and Telemetry	5
xrcomm_ip_client_get_dropped	5
xrcomm_ip_client_report_spectrum	5

Introduction

This document provides detailed guidelines for developers writing, compiling, and running custom applications that interface with the XRComm FCS 8T8R Platform.

The platform provides a header-only, zero-copy C API (`xrcomm_ip_client.h`) for POSIX applications that consume I/Q sample batches.

Compiling and Running the Hello World Example

The platform includes an example consumer application in the `XRComm_8T8R_hello_world/` folder that demonstrates how to subscribe to and consume platform data streams from specific ports and channels.

Compile the Example

From the `XRComm_FCS_platform` root directory:

```
cd XRComm_8T8R_hello_world
mkdir -p build && cd build
cmake ..
make
cd ../../
```

Compiled binaries are placed in `XRComm_8T8R_hello_world/bin/`.

Run the IQ Example

The IQ example connects to the Platform Driver via zero-copy shared memory. It requests access to the shared memory ring buffer and prints received batch counts and calculated power levels (dBFS).

Terminal 1 (FlexServer):

```
./XRComm_8T8R_hello_world/bin/XRComm_8T8R_hello_world
```

Terminal 2 (Client Computer or FlexServer):

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-ip on
```

Verify the Application Is Connected

Query the registry from the **Client Computer**:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 list-ips
# Output: slot=0 hello_world mode=IQ active
```

Application API Reference

Include `<xrcomm/xrcomm_ip_client.h>` in your C/C++ applications. The header uses standard POSIX shared memory (`shm_open`, `mmap`) and does not require linking against DPDK libraries.

Lifecycle and Connection Functions

xrcomm_ip_client_init

```
int xrcomm_ip_client_init(xrcomm_ip_client_t *client, const char *name);
```

Registers the application with the platform under the specified name and maps the primary control region. Returns 0 on success.

xrcomm_ip_client_init_port

```
int xrcomm_ip_client_init_port(xrcomm_ip_client_t *client, const char *name,
                               uint32_t hsp_port, uint32_t channel);
```

Registers the application and requests a specific NIC port (0 or 1) and channel (0..3, or 255 for all channels combined).

xrcomm_ip_client_wait_ready

```
int xrcomm_ip_client_wait_ready(xrcomm_ip_client_t *client, int timeout_ms);
```

Blocks the caller until the Platform Driver has created and opened the shared memory ring buffer for this application.

xrcomm_ip_client_running

```
bool xrcomm_ip_client_running(xrcomm_ip_client_t *client);
```

Returns `true` as long as the platform's main receive loop is active. Use this function as the loop guard for sample ingestion.

xrcomm_ip_client_shutdown

```
void xrcomm_ip_client_shutdown(xrcomm_ip_client_t *client);
```

Deregisters the application from the platform and unmaps/unlinks all shared memory regions.

Data Consumer Functions

xrcomm_ip_client_poll

```
const xrcomm_iq_slot_t* xrcomm_ip_client_poll(xrcomm_ip_client_t *client);
```

Polls the application ring buffer for a new batch of IQ samples. Returns a read-only pointer to the sample slot if data is available, or `NULL` if the ring is empty.

xrcomm_ip_client_consume

```
void xrcomm_ip_client_consume(xrcomm_ip_client_t *client);
```

Releases the last slot returned by `poll()`, advancing the read pointer and returning the slot to the Platform Driver's write pool. Must be called exactly once after a successful poll.

Diagnostics and Telemetry

xrcomm_ip_client_get_dropped

```
uint64_t xrcomm_ip_client_get_dropped(xrcomm_ip_client_t *client);
```

Returns the number of sample batches dropped by the Platform Driver since connection start due to the application's ring buffer being full.

xrcomm_ip_client_report_spectrum

```
void xrcomm_ip_client_report_spectrum(xrcomm_ip_client_t *client,  
                                     uint64_t processed, float power_dbfs);
```

Reports the application's processing metrics (total batches consumed and current signal power) back to the Platform Driver. These values are displayed in the gRPC telemetry and statistics.